

Technical Documentation

AI Service Desk — Ticket Management API

Date: July 24, 2026, **Version:** 1.0,

Revision History

Version	Date	Author	Description
1.0	24-Jul-2026	Diwakar	First complete release: full API reference, data model, test strategy, and deployment guide.

Table of Contents

Technical Documentation	1
1. Introduction	1
1.1 Purpose.....	1
1.2 Scope.....	1
1.3 Intended Audience.....	1
1.4 References	2
1.5 Acronyms and Definitions	2
2. System Overview.....	3
2.1 High-Level Architecture.....	3
2.2 System flow - Key Components	4
2.3 User Roles and Access Levels	6
3. Functional Requirements	7
3.1 Module/Feature Name.....	7
3.2 Inputs/Outputs.....	7
3.3 Business Rules	8
3.4 Error Handling.....	8
4. Non-Functional Requirements	9
4.1 Performance	9
4.2 Security.....	10
4.3 Availability	10
4.4 Maintainability	11
5. Data Model / Database Design	11
5.1 Entity-Relationship Diagram	11
5.2 Tables and Fields	12
5.3 Indexes and Keys	13
5.4 Sample Data	13
6. Integration Points	14
6.1 External Systems	14
6.2 APIs Used	14
6.3 Data Flow Diagram	15
6.4 Authentication/Authorization	16

7. Test Strategy	16
7.1 Test Scenarios	16
7.2 Expected Results.....	17
7.3 Traceability to Requirements.....	18
8. Deployment & Configuration	18
8.1 Environments.....	18
8.2 Setup Steps.....	19
8.3 Configuration Files and Parameters	20
9. Appendix.....	21
Glossary	21
Screenshots	21
Code Snippets	22
References	23

1. Introduction

1.1 Purpose

This document is the Technical Documentation for the AI Service Desk — Ticket Management API, a backend service that lets support teams create, track, and resolve customer tickets, and that uses Amazon Bedrock foundation models to draft ticket summaries and suggested agent responses. It was created to give engineers, reviewers, and academic evaluators a single, authoritative reference for how the system is designed, built, tested, and deployed, so that the codebase can be understood, maintained, and extended without relying on tribal knowledge.

Specifically, this document aims to:

- Describe the system's architecture and the responsibilities of each layer.
- Specify the functional and non-functional requirements the implementation satisfies.
- Document the database schema, API contract, and integration with AWS Bedrock.
- Record the design decisions made and the rationale behind each technology choice.
- Describe the test strategy, deployment procedure, and environment configuration needed to run the system reliably.

1.2 Scope

The scope of this document covers the AI Service Desk backend service exactly as implemented in the accompanying source repository: the FastAPI application, its service/repository/data layers, the PostgreSQL schema, the Alembic migration workflow, the Bedrock-based AI summarization feature, the automated test suite, and the Docker-based deployment stack.

Out of scope for this document:

- A dedicated front-end or mobile client — the project currently exposes a REST API and auto-generated Swagger/ReDoc documentation only; no browser-based UI ships with the repository.
- Production-grade identity and access management (SSO, OAuth2 login flows) — the current codebase does not implement endpoint-level authentication; this is called out explicitly in Sections 2.3 and 6.4 together with recommendations.
- Cloud infrastructure-as-code (Terraform/CloudFormation) for provisioning AWS resources — the project assumes AWS credentials and a Bedrock-enabled account are supplied externally via environment variables.

1.3 Intended Audience

This document is intended for the following stakeholders:

Audience	How this document helps them
Backend Developers	Understand the layered architecture, coding conventions, and how to extend the ticket and AI summarization features.
QA / Test Engineers	Reference the test strategy, existing test suite, and traceability matrix when writing new test cases.
DevOps / SRE	Follow the deployment, configuration, and environment-variable reference to provision and operate the service.
Product Owners	Review functional requirements and API capabilities to plan future iterations.
Academic Reviewers / Evaluators	Assess the design decisions, architecture, and engineering practices used in the project.

1.4 References

The following documents, tools, and external resources are relevant to this project:

- Project README.md — repository-level overview, setup instructions, and API reference.
- FastAPI official documentation — <https://fastapi.tiangolo.com>
- SQLAlchemy 2.0 (async) documentation — <https://docs.sqlalchemy.org>
- Alembic documentation — <https://alembic.sqlalchemy.org>
- Pydantic v2 documentation — <https://docs.pydantic.dev>
- Amazon Bedrock / Converse API documentation — <https://docs.aws.amazon.com/bedrock>
- Docker & Docker Compose documentation — <https://docs.docker.com>
- pytest and pytest-cov documentation — <https://docs.pytest.org>
- Locust load-testing documentation — <https://docs.locust.io>
- uv (Python package manager) documentation — <https://docs.astral.sh/uv>

1.5 Acronyms and Definitions

The table below defines the acronyms and domain-specific terms used throughout this document.

Term	Definition
API	Application Programming Interface — the REST interface exposed by the FastAPI application.
ORM	Object-Relational Mapper — SQLAlchemy, used to map Python objects to PostgreSQL rows.
CRUD	Create, Read, Update, Delete — the four basic persistence operations.

DTO	Data Transfer Object — implemented as Pydantic schemas for request/response validation.
UUID	Universally Unique Identifier — the primary-key type used for the tickets table.
E2E	End-to-End test — a test that exercises the full request lifecycle against a live database.
Bedrock	Amazon Bedrock — AWS's managed service for accessing foundation (large language) models.
Converse API	A unified Bedrock API for conversational/completion requests across model providers.
YAML	YAML Ain't Markup Language — the format used to externalize versioned AI prompt templates.
Alembic	A database migration tool for SQLAlchemy that manages schema version history.
CI	Continuous Integration — automated build/test execution on code changes.
RBAC	Role-Based Access Control — an authorization model referenced in Sections 2.3 and 6.4.

2. System Overview

2.1 High-Level Architecture

The AI Service Desk is a single-service, layered backend built on FastAPI. Client applications communicate with the API exclusively over HTTP/HTTPS; the API in turn depends on a PostgreSQL database for persistence and on Amazon Bedrock for AI-generated ticket summaries. The diagram below (produced from the Mermaid definition beneath it) shows the high-level component architecture.

```
graph TD
    Client[Client / HTTP REST] -->|HTTPS| App[FastAPI Application]
    subgraph App[FastAPI Application]
        MW[Middleware: CORS, Request Logging]
        EH[Exception Handlers: SQLAlchemyError, Global]
        RT[API Routers: ticket.py, ai.py]
    end
    App --> SVC[Service Layer: ticket_service, bedrock_services]
    SVC --> REPO[Repository Layer: ticket_repository.py]
    REPO --> DB[(PostgreSQL 17: tickets table)]
    SVC -->|AI summarize| BR[AWS Bedrock: Converse API]
```

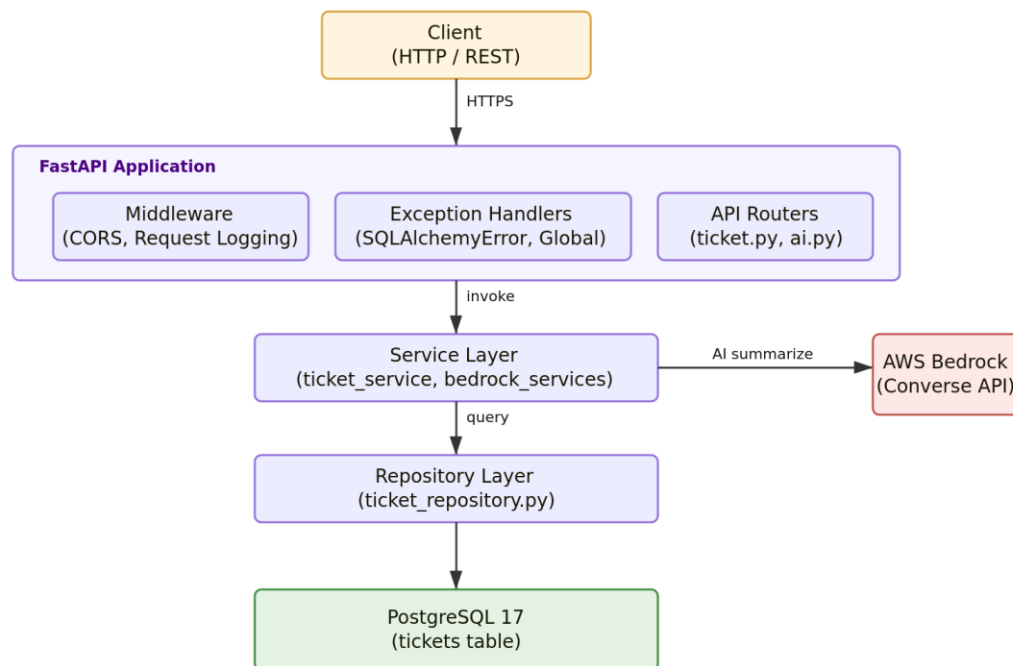


Figure 2.1 — High-level component architecture

Design rationale for the overall architecture:

- FastAPI was selected for its native async support, automatic OpenAPI/Swagger generation, and first-class Pydantic integration, which keeps request/response validation declarative and close to the route definitions.
- The application follows a strict Repository Pattern (API → Service → Repository → ORM → Database) so that business logic, persistence concerns, and HTTP concerns remain independently testable and replaceable.
- PostgreSQL was chosen over a lighter-weight database (e.g. SQLite) because it supports native ENUM types, UUID generation, and is the de-facto standard for production-grade relational workloads; the async driver asyncpg keeps the data layer non-blocking.
- Amazon Bedrock was chosen over a direct OpenAI/Anthropic SDK integration so the team can swap foundation models (Claude, Titan, Llama, etc.) through configuration (BEDROCK_MODEL_ID) without changing application code, and so that AI usage stays inside the same AWS account boundary as the rest of the infrastructure for billing and IAM governance.

2.2 System flow - Key Components

Each incoming request flows through five well-defined stages. The sequence diagram below illustrates the request/response lifecycle for a representative write operation, POST /tickets/.

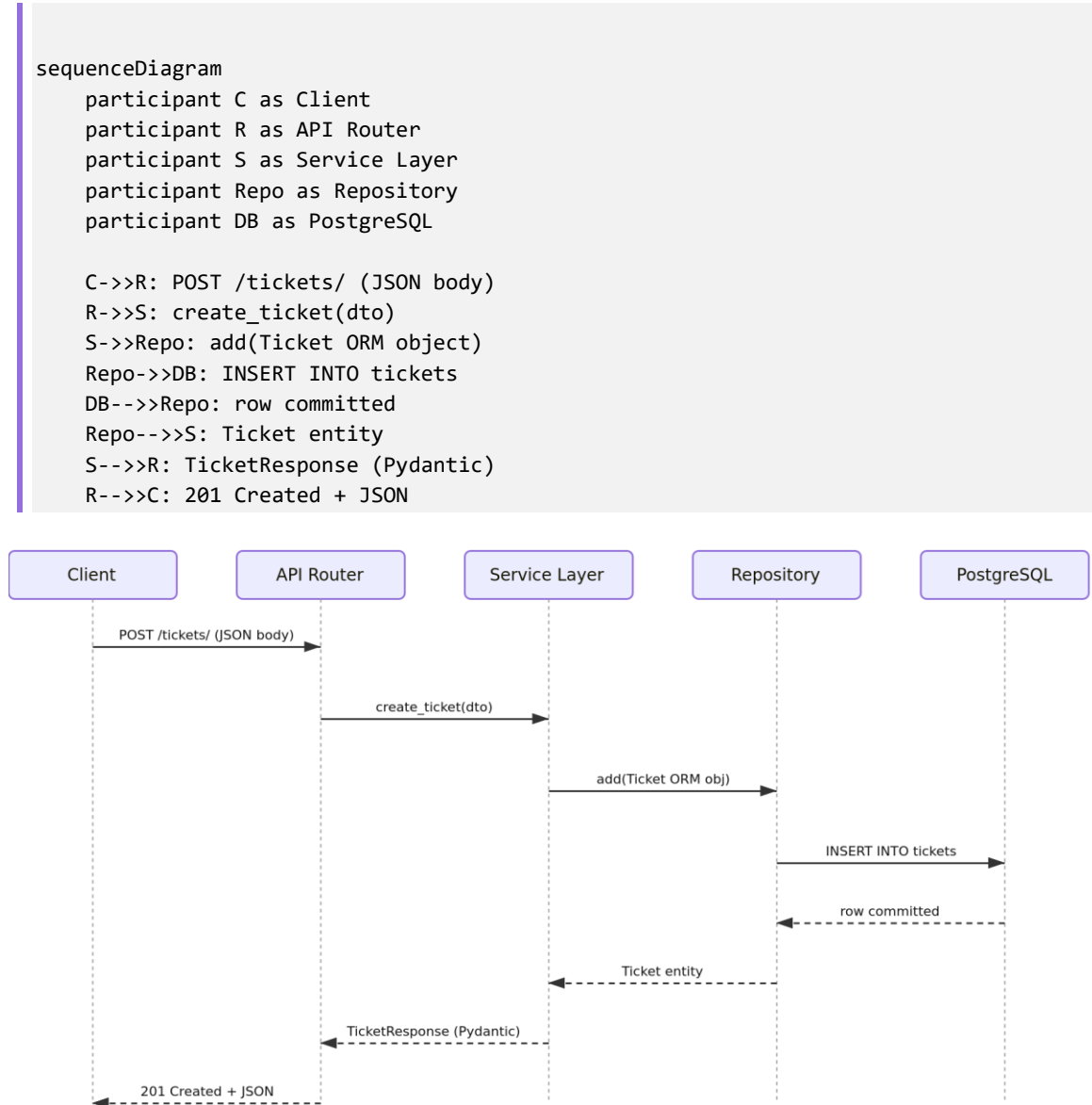


Figure 2.2 — Request flow for ticket creation

Key components and their responsibilities:

Component	Location	Responsibility
Middleware	app/main.py	Applies CORS policy and logs request duration for every call.
Exception Handlers	app/main.py	Catches SQLAlchemyError and unhandled exceptions and converts them to clean JSON error responses.
API Routers	app/api/ticket.py, app/api/ai.py	Declare routes, parse/validate request data via Pydantic, and return typed responses.
Service Layer	app/services/ticket_service.py, bedrock_services.py	Implements business logic: orchestrates repository calls and, for AI features, calls Amazon Bedrock.
Repository Layer	app/repositories/ticket_repository.py	Encapsulates all SQLAlchemy queries; the only layer aware of ORM/session details.
Schema Layer	app/schemas/ticket.py	Pydantic models that validate inbound payloads and shape outbound JSON.
Model Layer	app/models/ticket.py	SQLAlchemy declarative ORM model mapped to the tickets table.
Core Layer	app/core/config.py, deps.py	Loads settings from .env and provides FastAPI dependency-injected DB sessions/repositories.

2.3 User Roles and Access Levels

Applicability note: the current implementation does not enforce role-based access control at the endpoint level — every route under /tickets and /ai is reachable by any caller that can reach the service, and the assignee_email field on a ticket is a free-text attribute rather than an authenticated identity. This is an intentional simplification for the training/demo scope of the project (see Section 6.4 for the related authentication discussion). The table below documents the conceptual roles the domain model anticipates, which would be enforced by a future authentication layer.

Role	Description	Anticipated Permissions
Requester	The end customer who reports an issue.	Create tickets; view the status of their own tickets.
Support Agent	Identified today only via the assignee_email field.	View, update status/priority of assigned tickets; trigger AI summarization.

Administrator	Operates the service and manages configuration.	Full CRUD on all tickets; manage environment configuration and prompt templates.
System / Service Account	Used by health checks and orchestration tooling.	Read-only access to /health and /ready.

3. Functional Requirements

3.1 Module/Feature Name

The system is organized into three functional modules, each exposed through its own router and backed by a dedicated service class.

Module	Purpose	Key Files
Ticket Management	Full CRUD lifecycle for support tickets: create, retrieve (single/paginated/filtered), update, and delete, with UUID identification.	app/api/ticket.py, ticket_service, ticket_repository.py
AI Summarization	Accepts free-text ticket descriptions and returns an AI-generated summary plus a suggested agent response, using Amazon Bedrock's Converse API (or a deterministic FakeBedrockService in demo mode).	app/api/ai.py, bedrock_services, prompt_templates.py/.yaml
Platform / Operations	Root, health, and readiness endpoints used for smoke testing and container orchestration probes.	app/main.py

Ticket Management is the core module: every ticket is persisted with a title, priority, status, creation timestamp, and optional assignee e-mail. AI Summarization is a stateless, read-through capability layered on top — it does not persist results; the summary and suggested response are returned directly to the caller and are expected to be attached to a ticket by the calling client if desired.

3.2 Inputs/Outputs

The table below defines the input and output contract for every endpoint.

Endpoint	Input	Output
GET /	None	{message, version} — application name and API version.

GET /health	None	{status, service, version}; HTTP 503 if the database is unreachable.
GET /ready	None	{status: "ready"} once the database accepts connections.
POST /tickets/	JSON: title, priority, assignee_email	201 Created + full Ticket JSON (id, status, created_at, is_resolved, ...).
GET /tickets/	Query: status, priority, skip, limit (1-100, default 50)	Paginated list of Ticket JSON objects.
GET /tickets/{id}	Path: ticket UUID	Ticket JSON, or 404 if not found.
PUT /tickets/{id}	JSON (all optional): title, priority, status, assignee_email	Updated Ticket JSON, or 404 if not found.
DELETE /tickets/{id}	Path: ticket UUID	{message: "Ticket deleted successfully"}, or 404 if not found.
POST /ai/summarize	JSON: ticket_description (free text)	{summary, suggested_response} generated via Bedrock or the demo fake service.

3.3 Business Rules

The following business rules and validation constraints are enforced by the Pydantic schema and service layers:

- title must be between 3 and 200 characters (enforced by Pydantic field constraints on TicketCreate/TicketUpdate).
- priority is restricted to the enumeration low | medium | high and defaults to medium when omitted.
- status is restricted to open | in_progress | resolved | closed and defaults to open on creation; updates may transition to any of the four values (no explicit state machine is enforced — this is called out as a possible future enhancement).
- assignee_email is optional and, when supplied, must be a syntactically valid e-mail address (Pydantic EmailStr-style validation).
- id is server-generated (uuid4) and immutable; clients cannot set it on create.
- created_at is set by the database (server_default now()) and is never client-supplied.
- Pagination limit is capped at 100 records per page (skip/limit query parameters) to protect the service from unbounded result sets.
- AI summarization falls back to a deterministic FakeBedrockService whenever AWS_DEMO_MODE=true, so the /ai/summarize endpoint is fully testable without live AWS credentials or network access.

3.4 Error Handling

Exceptions are handled centrally rather than inside individual route handlers, keeping error formatting consistent across the API.

Condition	HTTP Status	Handling Mechanism
Pydantic validation failure (bad request body/query params)	422	FastAPI's built-in RequestValidationError handler returns a structured field-level error list.
Ticket not found (GET/PUT/DELETE by id)	404	Raised explicitly in the service layer as HTTPException(404) with a descriptive message.
Database error (constraint violation, connection failure, etc.)	500	Global SQLAlchemyError handler in app/main.py catches the exception, logs it, and returns a clean JSON error body instead of leaking a stack trace.
Any other unhandled exception	500	A catch-all global exception handler logs the error request context and returns a generic JSON error response.
Database unreachable at startup/runtime	503	GET /health reports unhealthy status; container orchestrators can use this to restart or hold traffic from the instance.

All handled exceptions are recorded through the application's structured logging middleware (request duration, path, and status code), which gives operators a single place to correlate failures with the requests that triggered them.

4. Non-Functional Requirements

4.1 Performance

The service is built end-to-end on SQLAlchemy's async engine and asyncpg, so a single Uvicorn worker can serve concurrent requests without blocking on database I/O. Performance is measured, not assumed, using two complementary tools documented in Section 7: cProfile for CPU-level profiling of individual endpoint flows, and Locust for HTTP load testing under concurrent virtual users.

Aspect	Target / Expectation
Simple read (GET /health, GET /ready)	Sub-10 ms typical response time; no database write involved.
Ticket CRUD operations	Single async round-trip to PostgreSQL per operation; response time dominated by network/DB latency rather than application logic.

List endpoint pagination	limit capped at 100 rows per request to bound query and payload size under load.
AI summarization (/ai/summarize)	Latency dominated by the Bedrock model invocation; AWS_DEMO_MODE=true removes this dependency entirely for load testing the API surface.
Concurrent load	Locust load tests (Section 7) exercise weighted mixes of all endpoints to validate the service holds up under realistic concurrent traffic.

Design rationale: an async stack was chosen specifically because ticket-desk workloads are I/O-bound (database round-trips, outbound Bedrock calls) rather than CPU-bound, so releasing the event loop during I/O gives materially better throughput per instance than a synchronous WSGI stack for the same hardware footprint.

4.2 Security

Security is addressed at three levels: input validation, secrets management, and data-access hygiene. The table below is deliberately candid about what is and is not implemented today, since an accurate security posture is more useful to reviewers than an aspirational one.

Concern	Current State	Notes / Recommendation
Input validation	Implemented	All request bodies and query parameters are validated by Pydantic v2 schemas before reaching business logic, preventing malformed or oversized payloads from propagating.
SQL injection	Mitigated	All queries go through SQLAlchemy's ORM/Core query builder with bound parameters; no raw string-interpolated SQL is used.
Secrets management	Partial	Credentials (POSTGRES_PASSWORD, AWS keys, SECRET_KEY) are supplied via .env / environment variables and excluded from version control via .gitignore; a production deployment should source these from a managed secret store (e.g., AWS Secrets Manager) instead of a flat file.
Endpoint authentication/authorization	Not implemented	No API keys, session tokens, or OAuth2 flows currently gate access to /tickets or /ai routes. SECRET_KEY exists in configuration but is not yet wired into an auth mechanism. See Section 6.4 for the recommended path forward.
Transport security	Deployment-dependent	The application itself speaks plain HTTP; TLS termination is expected to be handled by a reverse

		proxy or load balancer in front of the container in any non-local deployment.
AWS access	Implemented (IAM)	Bedrock calls use standard AWS credential resolution (access key/secret or IAM role), scoped to the bedrock:InvokeModel / Converse permissions needed.

4.3 Availability

The application exposes dedicated liveness/readiness signals and is designed to run under a container orchestrator that can restart unhealthy instances automatically.

- GET /health verifies both API and database connectivity and returns HTTP 503 when the database is unreachable, allowing orchestrators to treat the instance as unhealthy.
- GET /ready confirms the database is ready to accept traffic, suitable for a Kubernetes-style readiness probe or a load-balancer health check before routing live traffic to a newly started container.
- The docker-compose.yml stack defines a PostgreSQL health check (pg_isready) so the web container's entrypoint script only proceeds once the database is actually accepting connections, avoiding startup race conditions.
- The entrypoint script polls the database for up to 30 retries before failing fast, trading a bounded startup delay for resilience against slow database cold starts.
- Redundancy and failover for PostgreSQL itself (replication, automated backups) are the responsibility of the hosting environment (e.g., Amazon RDS Multi-AZ) and are outside the scope of the application code.

4.4 Maintainability

Several concrete practices keep the codebase easy to change safely:

Practice	How it Helps
Layered / Repository architecture	Business logic, persistence, and HTTP concerns are isolated, so a change to one layer (e.g., swapping PostgreSQL for another RDBMS) has minimal blast radius.
Alembic migrations	Every schema change is captured as a reviewable, versioned migration script rather than an ad-hoc manual DDL change.
Automated test suite (30 tests, 79% coverage)	Unit and integration/E2E tests catch regressions before they reach production; coverage reports highlight untested paths.
Structured logging with request duration	Makes production issues diagnosable without attaching a debugger.

Externalized YAML prompt templates	AI prompt wording can be revised and versioned without a code deployment.
pydantic-settings configuration	All configuration is centralized and typed in app/core/config.py, avoiding scattered os.environ lookups.
uv dependency management with a lock file	Guarantees reproducible installs across developer machines and CI.

5. Data Model / Database Design

5.1 Entity-Relationship Diagram

The current data model consists of a single entity, tickets, with no foreign-key relationships to other tables. This reflects the project's scope: ticket ownership is tracked via a plain e-mail string rather than a normalized users table, since user/identity management is explicitly out of scope (Section 1.2). The ER diagram below therefore shows attribute-level detail for the single entity rather than a multi-table relationship map.

```
erDiagram
    TICKETS {
        UUID id PK
        VARCHAR_200 title
        ENUM priority "low | medium | high"
        ENUM status "open | in_progress | resolved | closed"
        TIMESTAMPTZ created_at
        VARCHAR_255 assignee_email
    }
```

TICKETS		
id	UUID	PK
title	VARCHAR(200)	NOT NULL
priority	ENUM(low,medium,high)	DEFAULT medium
status	ENUM(open,in_progress, resolved,closed)	DEFAULT open
created_at	TIMESTAMPTZ	DEFAULT now()
assignee_email	VARCHAR(255)	NULLABLE

Single-entity schema — no foreign-key relationships in the current data model

Figure 5.1 — Entity-Relationship diagram for the tickets entity

A future users/agents entity — referenced by a foreign key from tickets — is the natural next step once authentication (Section 6.4) is introduced; the single-entity model was a deliberate trade-off to keep the training project's scope focused on the API and AI-integration patterns rather than full identity management.

5.2 Tables and Fields

The tickets table stores all support-ticket data:

Column	Type	Constraints
id	UUID	Primary key, auto-generated (uuid4)
title	VARCHAR(200)	NOT NULL, 3-200 characters
priority	ENUM(low, medium, high)	Default: medium
status	ENUM(open, in_progress, resolved, closed)	Default: open
created_at	TIMESTAMP WITH TIME ZONE	Server default: now()
assignee_email	VARCHAR(255)	Nullable

The SQLAlchemy declarative model in `app/models/ticket.py` is the single source of truth for this schema; Alembic autogeneration (Section 5.3) derives migrations directly from it, keeping the model and the live database schema from drifting apart.

5.3 Indexes and Keys

Key and indexing strategy for the tickets table:

Key / Index	Column(s)	Purpose
Primary Key	id	Uniquely identifies each ticket; UUIDs avoid sequential-ID enumeration and simplify merging data across environments.
Implicit index	id (via PK)	PostgreSQL automatically indexes primary-key columns for fast point lookups (<code>GET /tickets/{id}</code>).
Recommended index	status	<code>GET /tickets/</code> supports filtering by status; an index here keeps list queries efficient as the table grows.

Recommended index	priority	Similarly supports the priority filter on the same endpoint.
Recommended composite index	(status, priority)	Optimizes the common combined filter (status=open&priority=high) used in triage dashboards.

The status and priority indexes are recommended additions rather than currently generated migrations; they are flagged here because the query patterns in Section 3.2 (GET /tickets/?status=...&priority=...) would benefit from them once ticket volume grows beyond a small dataset.

5.4 Sample Data

Applicable — the table below provides representative, non-sensitive example records resembling the shape of production data returned by the API.

id	title	priority	status	assignee_email
a1b2c3d4-...-90	Database connection timeout	high	open	admin@example.com
b7e2f9a1-...-11	Password reset email not received	medium	in_progress	support1@example.com
c93df102-...-45	Add dark mode to dashboard	low	resolved	dev@example.com
d04ea881-...-77	Ticket summarization returns 500	high	closed	support2@example.com

These records mirror the request/response examples already documented for POST /tickets/ in Section 3.2, and can be inserted directly through that endpoint for demonstration or manual testing purposes.

6. Integration Points

6.1 External Systems

The application integrates with the following external / supporting systems:

System	Type	Purpose
Amazon Bedrock	External cloud AI service	Generates ticket summaries and suggested agent responses via the Converse API against a configurable foundation model (BEDROCK_MODEL_ID).
PostgreSQL 17 (Alpine)	Data store (containerized)	System of record for all ticket data.

Adminer	Development/ops tool (containerized)	Web-based database browser for inspecting and manually editing data during development.
Docker Hub / container registry	Build/deploy infrastructure	Source of base images (python:3.13-slim, postgres:17-alpine, adminer:latest, ghcr.io/astral-sh/uv).

6.2 APIs Used

The service both exposes and consumes APIs:

APIs exposed by this service (see Section 3.2 for the full contract):

- General: GET /, GET /health, GET /ready
- Ticket Management: POST, GET, PUT, DELETE under /tickets/
- AI: POST /ai/summarize
- Auto-generated interactive documentation: GET /docs (Swagger UI), GET /redoc (ReDoc), GET /openapi.json (machine-readable OpenAPI schema)

APIs consumed by this service:

- AWS Bedrock Converse API (via the boto3 SDK) — invoked from bedrock_services.py to obtain AI-generated summaries; requests are built from the YAML prompt templates described in Section 8.3 and the README's Prompt Templates section.

6.3 Data Flow Diagram

The diagram below traces how data moves between the client, the application, the database, and Amazon Bedrock for the two representative flows: a standard ticket write and an AI summarization request.

```

flowchart LR
    C((Client)) -- 1. JSON request --> API[FastAPI Router]
    API -- 2. validated DTO --> SVC[Service Layer]
    SVC -- 3a. persist --> REPO[Repository]
    REPO -- 4a. SQL --> DB[(PostgreSQL)]
    SVC -- 3b. ticket_description --> PROMPT[Prompt Template Loader]
    PROMPT -- 4b. rendered prompt --> BR[AWS Bedrock Converse API]
    BR -- 5b. summary + suggested_response --> SVC
    DB -- 4a. row --> REPO
    REPO -- 5a. entity --> SVC
    SVC -- 6. response DTO --> API
    API -- 7. JSON response --> C
  
```

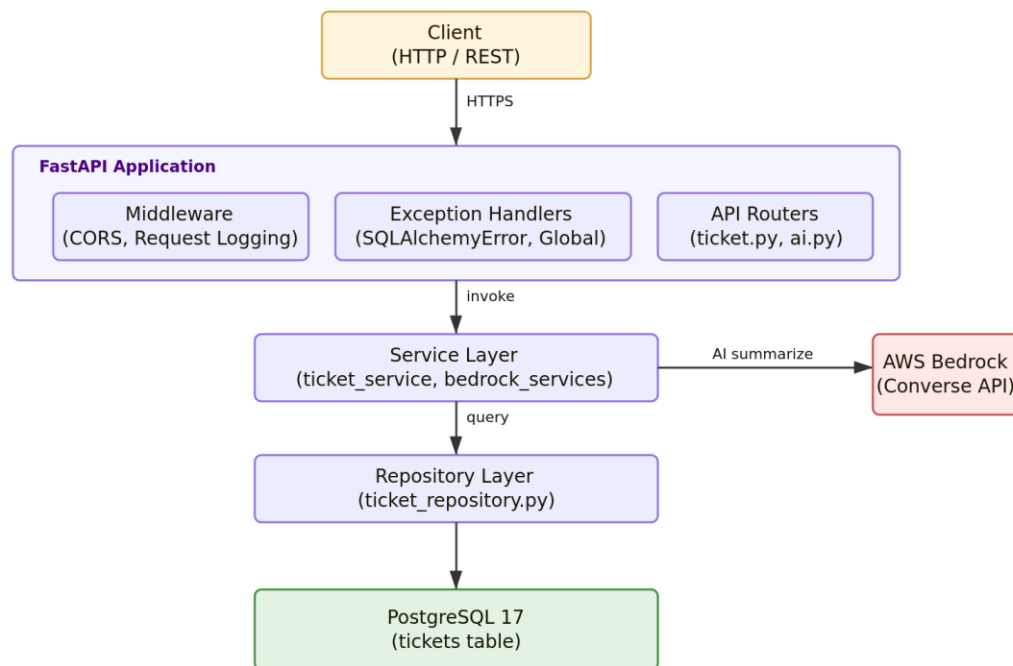


Figure 6.3 — Data flow reuses the same component boundaries shown in Figure 2.1; arrows above trace data direction rather than call direction.

In practice the two flows share the same entry point (the API router) and diverge inside the service layer: ticket writes flow down through the repository into PostgreSQL, while summarization requests flow out to Bedrock through the prompt template loader and never touch the database.

6.4 Authentication/Authorization

As noted in Sections 2.3 and 4.2, the API currently has no endpoint-level authentication or authorization — this is an explicit, documented limitation of the current scope rather than an oversight, appropriate for the project's training/demo context.

Two distinct authentication concerns exist in the system:

- Client-to-API authentication: not implemented. Any client that can reach the service's network can call every route. SECRET_KEY is present in configuration and reserved for a future JWT/session-based scheme but is not currently consumed by any middleware or dependency.
- API-to-AWS authentication: implemented via standard AWS credential resolution. AWS_ACCESS_KEY_ID and AWS_SECRET_ACCESS_KEY (or an attached IAM role in a

production/ECS/EKS deployment) authorize calls to Bedrock, scoped to the region set by AWS_REGION.

Recommended path to production-grade authorization:

1. Introduce an OAuth2/JWT bearer-token dependency (FastAPI's OAuth2PasswordBearer or an external identity provider) validated on every /tickets and /ai route.
2. Add a users/agents table (Section 5.1) and replace the free-text assignee_email with a foreign key to that table.
3. Enforce the RBAC roles already sketched in Section 2.3 (Requester, Support Agent, Administrator) as JWT claims checked by a FastAPI dependency.
4. Scope the AWS IAM role attached to the service to the minimum Bedrock permissions required (bedrock:InvokeModel for the configured model only).

7. Test Strategy

7.1 Test Scenarios

The automated suite comprises 30 tests split into three categories, executed with pytest and measured with pytest-cov.

File	Category	Count	Description
test_schemas.py	Unit	14	Pydantic schema validation: field constraints, enum values, computed fields (e.g. is_resolved).
test_services.py	Unit	6	Service-layer business logic exercised against a mocked (AsyncMock) repository.
test_routes.py	Integration & E2E	10	Full HTTP request lifecycle executed against a live test database.

Representative scenarios covered by the suite include:

- Creating a ticket with valid data returns 201 and a well-formed Ticket JSON body.
- Creating a ticket with a title shorter than 3 characters is rejected with 422.
- Listing tickets honors the status and priority query filters and the skip/limit pagination bounds.
- Requesting a non-existent ticket id returns 404 on GET, PUT, and DELETE.
- Updating a ticket with a partial payload only changes the supplied fields.
- GET /health returns 503 when the database dependency is unavailable (mocked failure).

- POST /ai/summarize returns the deterministic FakeBedrockService output when AWS_DEMO_MODE=true.
- The conftest.py clear_tickets fixture truncates the tickets table before each test so tests remain independent and repeatable.

Test fixtures (conftest.py):

Fixture	Purpose
clear_tickets	Truncates the tickets table before each test (autouse).
mock_repo	Provides an AsyncMock repository for isolated unit testing of the service layer.
existing_ticket_id	Creates a real ticket and returns its UUID for integration tests that need an existing record.

7.2 Expected Results

Pass/fail criteria for the suite as a whole and for coverage measurement:

- All 30 tests must pass (uv run pytest exits with status 0) before a change is considered mergeable.
- Each unit test asserts a specific field, status code, or return value rather than a broad smoke check, so a failure pinpoints the exact behavior that regressed.
- Coverage is collected automatically via pytest-cov (configured in pyproject.toml with --cov=app --cov-report=term-missing --cov-report=html) and currently stands at 79%, reported both in the terminal and as an interactive HTML report at htmlcov/index.html.
- Integration/E2E tests are expected to run against a real PostgreSQL instance (via docker compose up db -d) rather than a mock, so they also implicitly validate the Alembic-migrated schema.

7.3 Traceability to Requirements

The matrix below maps test files to the functional and non-functional requirements they validate.

Requirement (Section)	Validated By
3.2 Inputs/Outputs — request/response contracts	test_schemas.py, test_routes.py
3.3 Business Rules — field constraints & defaults	test_schemas.py
3.4 Error Handling — 404 / 422 / 503 behavior	test_routes.py
3.1 AI Summarization module	test_services.py (mocked Bedrock), test_routes.py (demo-mode E2E)

4.3 Availability — health/readiness probes	test_routes.py
5.2 Tables and Fields — schema correctness	test_routes.py (via live-database E2E writes/reads)

8. Deployment & Configuration

8.1 Environments

The project supports two primary environments today; a production topology would layer on top of the same container images with externalized secrets and a managed database.

Environment	How it's run	Purpose
Local Development	uv run uvicorn app.main:app --reload against a locally running or docker compose up db -d PostgreSQL instance	Fast iteration with hot reload; developers run migrations manually.
Docker Compose (integrated)	docker compose up --build -d, bringing up web, db, and db-ui together	Full-stack demo/QA environment that mirrors production topology on a single host.
Production (recommended target, not shipped)	Container images built from the same Dockerfile deployed to an orchestrator (ECS/EKS/etc.) with a managed PostgreSQL service and secrets sourced from a vault	Not part of this repository's scope; documented here as the natural extension of the existing Docker artifacts.

The diagram below shows the deployment/startup flow triggered by docker compose up, including the entrypoint script's three-step startup sequence.

```

flowchart TD
    Start[docker compose up --build -d] --> DBC[Container: t1_db - PostgreSQL 17 Alpine]
    DBC -->|healthcheck: pg_isready| WebStart[Container: t1_web starts - entrypoint.sh]
    WebStart --> UIC[Container: t1_db_ui - Adminer :8080]
    WebStart --> S1[1. Poll DB connection - up to 30 retries]
    S1 --> S2[2. alembic upgrade head - auto-migrate]
    S2 --> S3[3. uvicorn app.main:app - port 8000]
    S3 --> Ready[FastAPI service READY - GET /health returns 200]
  
```

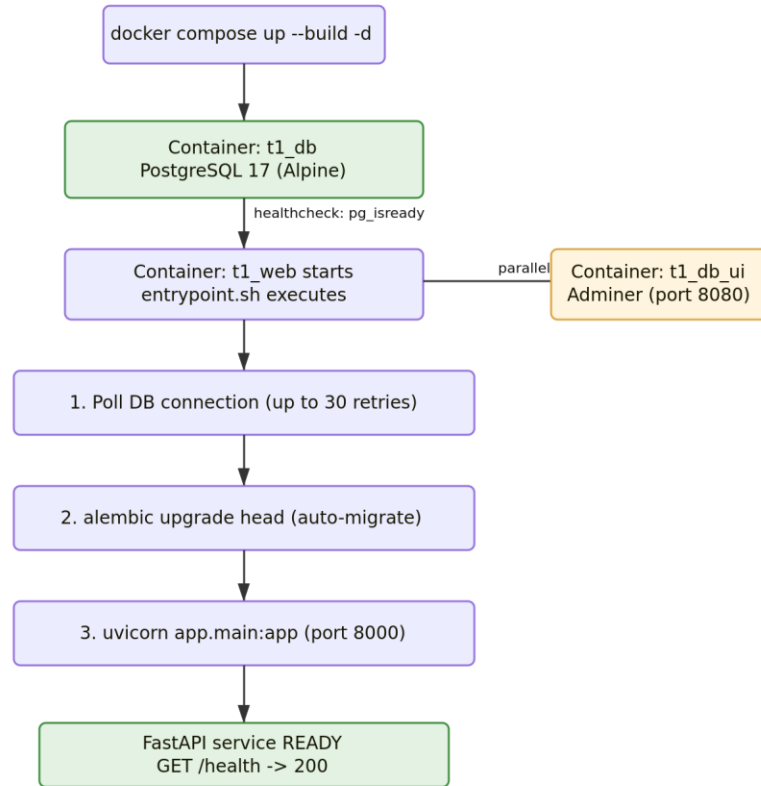


Figure 8.1 — Container startup / deployment flow

8.2 Setup Steps

Setup differs slightly between local development and the fully containerized stack.

Local Development:

```

git clone https://github.com/Diwakar0109/goml-python-day1.git
cd goml-python-day1
cp .env.example .env          # then edit DB / AWS credentials
uv sync                       # install dependencies
docker compose up db -d       # start PostgreSQL only
uv run alembic upgrade head    # apply migrations
uv run uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
  
```

The API becomes available at `http://localhost:8000`.

Docker Deployment (full stack):

```

docker compose up --build -d  # build + start web, db, db-ui
docker ps                    # verify all containers are running
docker logs -f t1_web        # follow application logs
  
```

As shown in Figure 8.1, the entrypoint script automatically polls the database, runs alembic upgrade head, and starts Uvicorn — no manual migration step is required in the containerized path.

8.3 Configuration Files and Parameters

Configuration is environment-driven. The table below documents every variable defined in `.env.example` / consumed by `app/core/config.py`.

Variable	Description	Default
APP_NAME	Application display name	t1
API_VERSION	API version string	v1
DEBUG	Enable debug mode	True
POSTGRES_USER	PostgreSQL username	postgres
POSTGRES_PASSWORD	PostgreSQL password	123
POSTGRES_DB	PostgreSQL database name	ai_service_desk
DATABASE_URL	Async database connection string	postgresql+asyncpg://postgres:123@db:5432/ai_service_desk
SECRET_KEY	Application secret key (reserved for future auth)	my-secret-key
AWS_ACCESS_KEY_ID	AWS access key for Bedrock	—
AWS_SECRET_ACCESS_KEY	AWS secret key for Bedrock	—
AWS_REGION	AWS region for Bedrock	us-east-1
AWS_DEMO_MODE	Use FakeBedrockService instead of live AWS calls	false
BEDROCK_MODEL_ID	Bedrock foundation model identifier	—

Note: when running inside Docker, the database host resolves to db (the Compose service name); when running locally, the application auto-detects and falls back to localhost. Also included is `app/services/prompt_templates.yaml`, a versioned configuration file (not an environment variable) that externalizes the AI prompt wording used by the summarization service, loaded once at import time by `load_prompt_template()`.

9. Appendix

Glossary

Key technical and business terms used throughout this document:

Term	Definition
Ticket	A single support request tracked by the system, identified by a UUID.
Demo Mode	A configuration flag (<code>AWS_DEMO_MODE=true</code>) that swaps live Bedrock calls for a deterministic <code>FakeBedrockService</code> , enabling offline development and classroom demonstrations.
Prompt Template	A versioned, YAML-defined text template rendered with <code>str.format()</code> to build the prompt sent to the Bedrock model.
Repository (pattern)	A design pattern that isolates all database-query logic behind a dedicated class, keeping the service layer persistence-agnostic.
Readiness Probe	A health-check endpoint (<code>GET /ready</code>) used by orchestrators to decide whether an instance should receive live traffic.
Liveness Probe	A health-check endpoint (<code>GET /health</code>) used by orchestrators to decide whether an instance should be restarted.
Coverage	The percentage of application code lines executed by the automated test suite, measured with <code>pytest-cov</code> .

Screenshots

Not applicable in the traditional sense: the AI Service Desk is a headless REST API with no bundled front-end, so there are no application screens to capture. In lieu of UI screenshots, the system's interactive surface is the auto-generated API documentation that FastAPI serves directly from the running application:

Documentation	URL
Swagger UI	http://localhost:8000/docs
ReDoc	http://localhost:8000/redoc
OpenAPI JSON	http://localhost:8000/openapi.json

Swagger UI in particular provides a fully interactive "try it out" experience for every endpoint documented in Section 3.2, and functionally supersedes static screenshots since it always reflects the current state of the deployed code. Adminer (<http://localhost:8080>,

described in Section 6.1) similarly provides a live, browsable view of the tickets table for anyone who needs a visual on the underlying data.

Code Snippets

A small number of representative, reusable code blocks are included below for reference.

1. Versioned prompt template (app/services/prompt_templates.yaml):

```
ticket_summary_v1:
  name: "ticket_summary"
  version: "1.0.0"
  template: >-
    Return only valid JSON with keys summary and suggested_response.
    Summarize the following support ticket and suggest a concise,
    professional response. Ticket: {ticket_description}
```

2. Rendering a prompt template at runtime:

```
from app.services.prompt_templates import TICKET_SUMMARY_V1

rendered = TICKET_SUMMARY_V1.render(
    ticket_description="User cannot login..."
)
```

3. AI summarization request/response contract:

```
# Request
POST /ai/summarize
{
  "ticket_description": "The database is crashing periodically under high load."
}

# Response
{
  "summary": "Database stability issue under high load requiring investigation.",
  "suggested_response": "We acknowledge the database instability and are
    investigating connection pooling and index optimization."
}
```

References

Links and citations for the documentation, third-party tools, and APIs referenced throughout this document:

- Project repository README.md
- FastAPI documentation — <https://fastapi.tiangolo.com>
- SQLAlchemy 2.0 documentation — <https://docs.sqlalchemy.org>
- Alembic documentation — <https://alembic.sqlalchemy.org>
- Pydantic v2 documentation — <https://docs.pydantic.dev>
- Amazon Bedrock documentation — <https://docs.aws.amazon.com/bedrock>

- boto3 (AWS SDK for Python) documentation — <https://boto3.amazonaws.com/v1/documentation/api/latest/index.html>
- Docker & Docker Compose documentation — <https://docs.docker.com>
- pytest documentation — <https://docs.pytest.org>
- Locust documentation — <https://docs.locust.io>
- uv documentation — <https://docs.astral.sh/uv>
- Adminer project — <https://www.adminer.org>